Hashing

IS2110 - Data Structures and Algorithms II

Kasun Karunanayaka
Senior Lecture, UCSC
ktk@ucsc.cmb.ac.lk
Thanks to Ms. Kokila Perera for original slides set

Hashing

- Introduction
  - Different Approaches for Searching
  - Direct-address Tables
  - Hash Tables
- Hash Functions
- Resolving Collisions
- Hashing applications

Dynamic Sets

Elements of Dynamic Sets

- **Keys:**
  - Some kinds of dynamic sets assume that one of the object's attributes is an identifying key.
  - If the keys are all different, we can think of the dynamic set as being a set of key values.
- **Satellite Data:**
  - The object may contain, which are carried around in other object attributes but are otherwise unused by the set implementation.

Operations:

- Search, insert, delete, minimum, maximum, successor, predecessor

Searching Problem

Given a collection C of elements, there are three fundamental queries one might ask:

- Does C contain a target element t?
- Return the element in C that matches the target element t
- Return information associated in collection C with the target key element k

Searching Algorithms

Sequential Search/Linear Search

The table that stores the elements is searched successively, and the key comparison determines whether or not an element has been found.

- Array, linked list

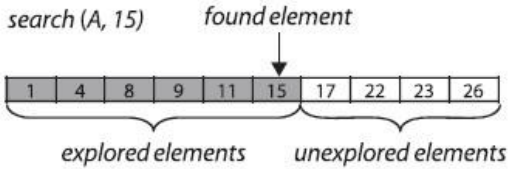
SEQUENTIAL SEARCH

Best	Average	Worst
O (1)	O (n)	O (n)

Array

Brute Force

```
search (A, t)
1.    for i = 0 to n − 1 do
2.       if (A[i] = t) then
3.          return true
4.    return false
end
```



Searching Algorithms

Binary search

- The table that stores the elements is divided successively into halves to determine which cell to be checked.
- The key comparison determines whether or not element has found.
- Search operation requires time proportional to the height of the tree in the worst case, which is O(log n) time in the average case over all trees, but O(n) time in the worst case.

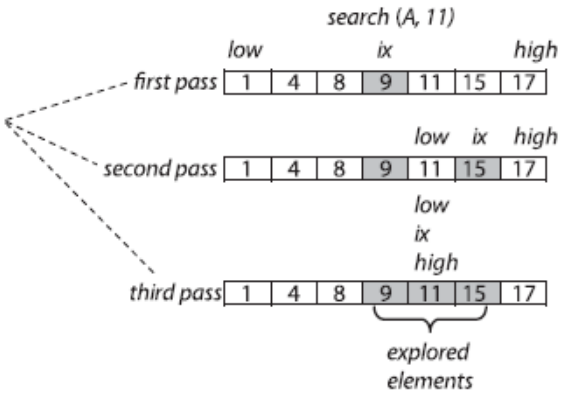
BINARY SEARCH

Best	Average	Worst
O (1)	O (log n)	O (log n)

Array

Divide and Conquer

```
search (A, t)
1.   low = 0
2.   high = n –1
3.   while (low ≤ high) do
4.     ix = (low + high)/2
5.     if (t = A[ix]) then
6.         return true
7.     else if (t < A[ix]) then
8.         high = ix – 1
9.     else low = ix + 1
10.  return false
end
```



Different Approaches for Searching

Jump Search

- Example:

(0, 1, 2, 5, 8, 10, 12, 15, 21, 34, 55, 89, 145, 232, 376, 681).

- Length of the array is 16. Block size to be jumped is 4.
- Find the value of 55.
- STEP 1: Jump from index 0 to index 4;
- STEP 2: Jump from index 4 to index 8;
- STEP 3: Jump from index 8 to index 12;
- STEP 4: Since the element at index 12 is greater than 55 requires jump back a step to come to index 8.
- STEP 5:Perform linear search from index 8 to get the element 55.

Different Approaches for Searching

Jump Search

- Works only sorted arrays.
- The optimal size of a block to be jumped is ($\sqrt{n}$). This makes the time complexity of Jump Search $O(\sqrt{n})$.
- The time complexity of Jump Search is between Linear Search and Binary Search
  - $O(\text{Log } n) \leq$ complexity of jump search $\leq O(n)$
- Jump search has an advantage that we traverse back only once (Binary Search may require up to O(log n) jumps, consider a situation where the element to be searched is the smallest element.
- So in a system where binary search is costly, we use Jump Search.

Different Approaches for Searching

Interpolation Search

- Works only on a sorted array.
- The **probe position calculation** is the only difference between binary search and interpolation search.
- It places a probe in a calculated position on each iteration. If the probe is right on the item we are looking for, the position will be returned; otherwise, the search space will be limited to either the right or the left side of the probe.

Different Approaches for Searching

Interpolation Search

- In binary search, the probe position is always the middlemost item of the remaining search space.
- In contrary, interpolation search computes the probe position based on this formula:

$$probe = lowEnd + \frac{(highEnd - lowEnd) \times (item - data[lowEnd])}{data[highEnd] - data[lowEnd]}$$

- Interpolation search time complexity varies depending on the data distribution.
- It is faster than binary search for uniformly distributed data with the time complexity of O(log(log(n))).

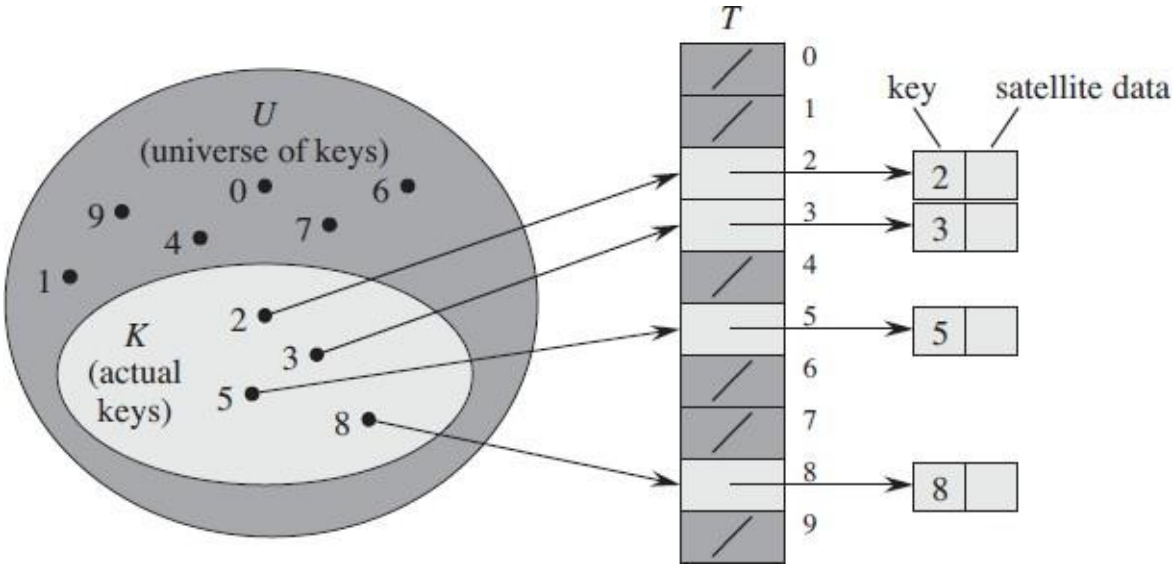
Different Approaches for Searching

- Calculates the position of the key in the table based on the value of the key (string, number, record, etc.).
- When the key is known the position in the table can be accessed directly, without making any other preliminary tests.
- The search time will be reduced to O(1).

Direct-address Tables

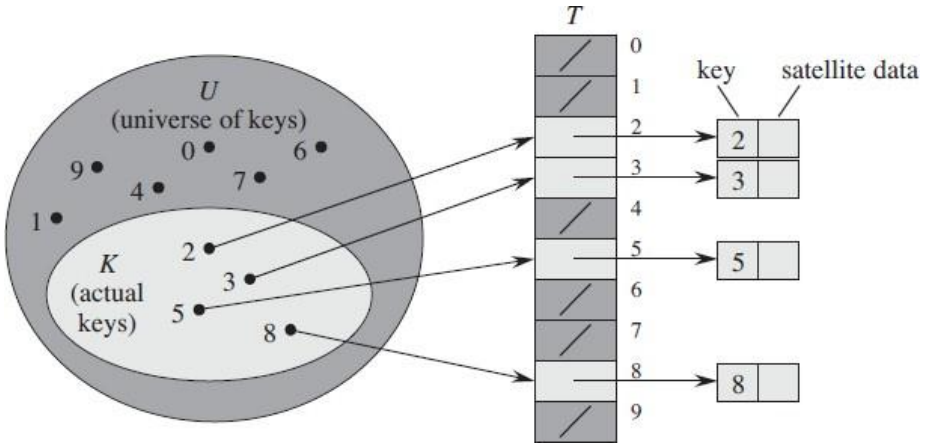
Dynamic-address table can be used to represent a dynamic set of m elements with **distinct** key values which are drawn from the universe U = {0, 1, …, m-1 }, where m is not too large.

Position of each key corresponds to the value of the key itself.



Direct-address Tables

- An array T[0 . . . m - 1]
- Each slot, or position, in T corresponds to a key in U
- For an element x with key k, a pointer to x (or x itself) will be placed in location T[k]
- If there are no elements with key k in the set, T[k] is empty, represented by NIL



Direct-address Tables

Main issue in direct address tables:

- If the universe U is large, storing a table of size |U| may be impractical or even impossible.
- The set of keys actually stored may be so small relative allocated space; most of the space allocated for T would be wasted.

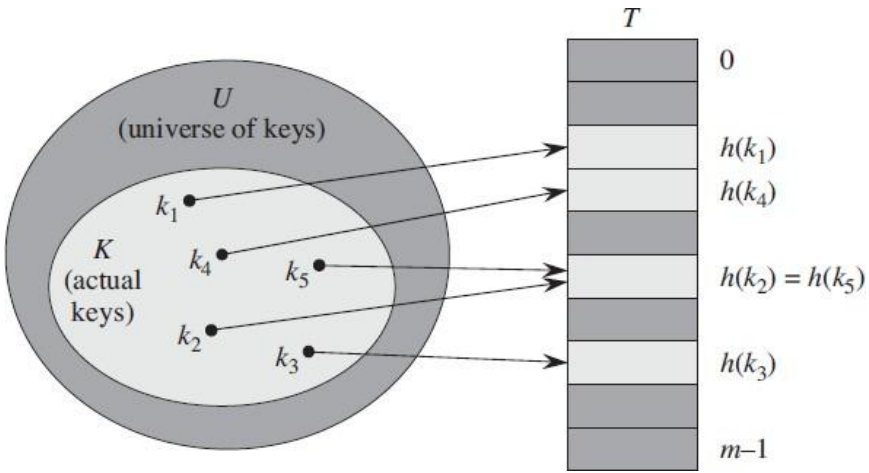
Hash Tables

- Many applications require a dynamic set that supports only the operations of INSERT, SEARCH, and DELETE.
- A hash table is an effective data structure for implementing dictionaries.
- Under reasonable assumptions, the average time to search for an element in a hash table is O(1) [in worst case O(n)].

Hash Tables

In hash tables T[0, 1, …, m-1], an element with the key k is stored in slot h(k)

- We use a **hash function h** to compute the slot for the element based on its key k.
- We say that an element with key k **hashes** to slot h(k) and **h(k)** is the **hash value** of key k

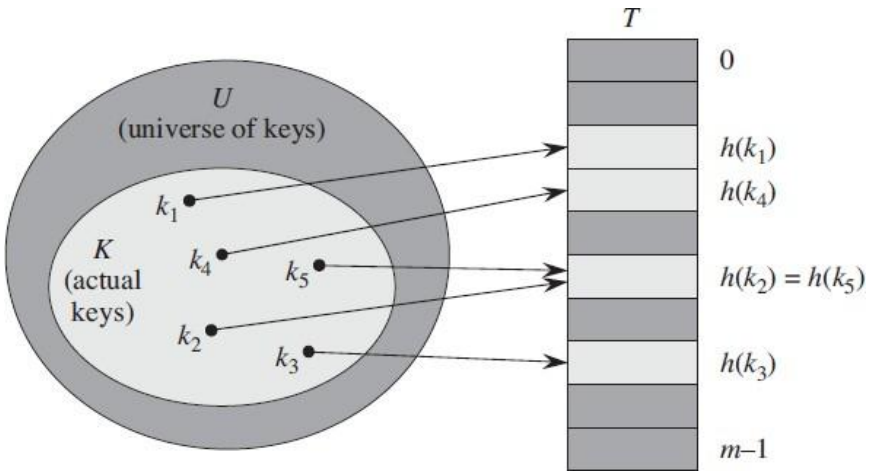


Hash Function

- The hash function transforms a key to an index in the hash table T[0, 1, …, m-1]

  H: U → {0, 1, …, m-1}

- Compared to direct-address tables hash tables have reduced range of array indexes handled and reduced storage



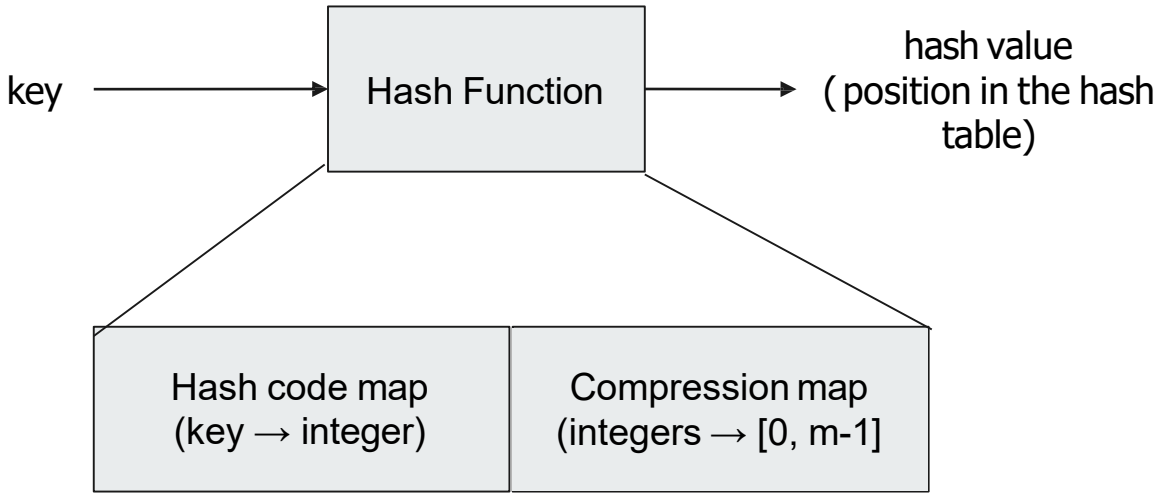
- m slots instead of |U|

Hash Function

- A good hash function satisfies (approximately) the assumption of simple uniform hashing:
  - Each key is equally likely to has to any of the m slots independently of where any other key hashes to.
  - The hash table should contain the same number of positions as the number of elements being hashed.
  - Typically, we do not know the probability distribution of the keys therefore there is no way to check this condition.
- Qualitative information about the distribution of keys are useful in designing a good hash function.
- Enables to trivially build an O(1) search time.

Hash Function

- Hash function (h) is a composition of two functions: h(key) = h2(h1(x))



Hash Function

- Hash function (h) is a composition of two functions:

$$h(key) = h2(h1(x))$$

- Hash code map: h1
  - h1: keys → integers
  - Integer cast
  - Summing components
  - Polynomial Accumulation

- Compression map: h2
  - H2: integers → [0, m-1]
  - Division method
  - Multiplication method
  - Mid-square function
  - Extraction
  - Radix transformation
  - Truncation
  - MAD method

Hash Function

Hash Code Map: Integer Cast

- Hash code that interprets the bits of the key as an integer.
- Suitable for keys of length less than or equal to the number of bits of the integer type (ex: byte, short, int, float in java)
- Does not work for long and double numeric types.

Hash Function

Hash Code Map: Integer Cast

Java Primitive Types	No. of bits
short	8
char	8
int	32
float	32
double	64
long	64

Ex:

```
int intkey =(int) 'A';          // int key = 65

char charKey =(char) 75;        // charKey ='K'
```

Hash Function

Hash Code Map: Summing Components

Ex: Applying to strings

$$word = (char(0), char(1), \ldots, char(k-1))$$

$$wordKey = h(word) = \sum_{i}^{k-1} (int)\ char(i)$$

- Word = "ALGO" = {'A, 'L', G', O'} $\Rightarrow$ $h1(word) = 65 + 76 + 71 + 79 = 291$
- Word = "HASH" = ?

Hash Function

Hash Code Map: Polynomial Accumulation

The polynomial : $x_0 + x_1 a + \ldots + x_{k-1} a^{k-1}$ where a is a constant of the hash code.

Horner's form:

$$x_0 + a(x_1 + a(x_1 + \ldots + a(x_{k-3} + a(x_{k-2} + a x_{k-1})) \ldots))$$

More efficient to use.

a is normally selected as 37 or 31.

Hash Function

Hash Code Map: Polynomial Accumulation

The polynomial : $x_0 + x_1 a + \ldots + x_{k-1} a^{k-1}$ **where a is a constant of the hash code.**

Horner's form:

$$x_0 + a(x_1 + a(x_2 + \ldots + a(x_{k-3} + a(x_{k-2} + a x_{k-1})) \ldots))$$

More efficient to use.

a is normally selected as 37 or 31.

t =>116, a=>97, p=>112

Example: "tap"

Hash Function

Compression Map: Division

$$h(k) = k \bmod m$$

- m is the size of hash table
- Usually the preferred choice for the hash function if very little is known about the keys.

Example:

- $K = \{44, 56, 79\}$
- $h: U \rightarrow \{0, 1, ..., 10\}$
- $h(k) = k \bmod 11$

Hash Function

Compression Map: Division

$$h(k) = k \bmod m$$

- Selecting m,
  - Avoid powers of 2
  - Avoid powers of 10 when decimal numbers are used as keys
  - A prime number not too close to an exact power of 2 is often a good choice.
    - A prime number helps to spread out the hash values.

Hash Function

Compression Map: Division

Example:

{200, 205, 210, 300, 305, 310, 400, 405, 410}

m = 100, hashed values {0, 5, 10, 0, 5, 10, 0, 5, 10}={0, 5, 10}

m = 101, hashed values hashed values {99, 3, 8, 98, 2, 7, 97, 1, 6}

When selecting a prime number as m, helps to spread out the hash values.

Hash Function

Compression Map: Multiplication Method

Multiply the key k by a constant A in the range $0 < A < 1$ and extract the fractional part which is then multiply this value by m,

$h(k) = \text{floor}(m(kA \bmod 1))$;
$= \text{floor}(8 ((100*0.9852) \bmod 1))$;
$= \text{floor}(8 (98.52 \bmod 1))$
$= \text{floor} (8 * 0.52)$
$= \text{floor}(4.16)$
$= 4$

Hash Function

Compression Map: Mid Square Function

The key is squared and the middle part/mid part of the result is used as the address. (If key is not numeric, it need to be mapped before hand)

Ex:

$$\text{key is } 2456 \Rightarrow \text{key}^2 = (2456)^2 = 6031936 \Rightarrow 60\mathbf{319}36$$

$$h(2456) = 319$$

Hash Function

Compression Map: Radix Transformation

The key is transformed into another number base.

Example: base 10 → 9

- If key is the decimal number 345, then its value in base 9 is 423.
- This value is then divided modulo m (size of hash table), and the resulting number is used as the address of the location to which K should be hashed.

Hash Function

Compression Map: Extraction

Only a part of the key is used to compute the address.

Example:

For the key 123-45-6789, this method might use

- the first four digits 1234
- select from middle 345678

Hash Function

Compression Map: Truncation

Use right most or left most n digits of the key.

Example: employee number : 123456789

- h(x) = (last three digits) = 789

Hash Function

Compression Map: Folding

Divide the key into several parts and perform an arithmetic operation (such as summation) on the parts.

Example: employee number : 123456789

- h(x) = (1st 3 digits) + (2nd 3 digits) + (3rd 3 digits)
  = 123 + 456 + 789
  = 1368

Universal Hashing

It is not recommended to use a fixed hashing algorithm because it can be vulnerable to malicious adversary attacks:

- A malicious adversary might chooses the keys to be hashed (by some fixed hash function) in a way that all keys hashed to the same slot → leads to average retrieval time of $\Theta(n)$.

Universal hashing can be used to overcome this issue.

Universal Hashing

In universal hashing, at the beginning of execution we select the hash function at random from a carefully designed class of functions.

Given a family H of hash functions and $M = \{0, 1, \ldots\ldots, m-1\}$ one function h from H is picked uniformly at random.

The algorithm can behave differently on each execution, even for the same input, guaranteeing good average-case performance for any input.

Universal Hashing

Let $\mathcal{H}$ be a finite collection of hash functions that map a given universe U of keys into the range
{0, 1, …, m-1}.

$\mathcal{H}$ is said to be universal if, with a hash function h randomly chosen from $\mathcal{H}$, the chance of a collision between distinct keys k and l is no more than the chance 1/m of a collision if h(k) and h(l) were randomly and independently chosen from the set {0, 1, …, m-1}

I.e.: for each pair of distinct keys k, l ϵ U, the number of hash functions h ϵ $\mathcal{H}$, for which h(k)=h(l) is at most $|\mathcal{H}|/m$.

Universal Hashing - Example

MAD Method (Multiply Add and Divide compression map)

Spreads n integers fairly evenly in the range [0, p-1]

- p is a prime number p > m and large enough that every possible key k is in the range [0, p -1], inclusive
- a ϵ {1, 2, …, p -1}
- b ϵ {0,1, …, p-1}

$$h(k) = [(ak + b) \bmod p] \bmod m$$

- Size of hash table: m not necessarily prime

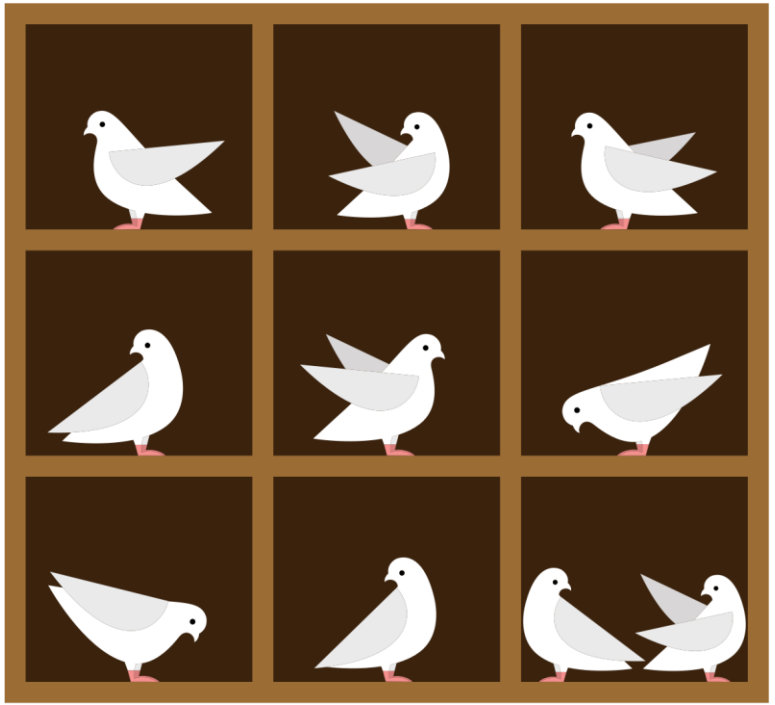
Collision

Two keys may hash to the same slot in the hash table.

1. Avoid collisions: choosing a suitable hash function

Pigeonhole Principle

If n items are put into m pigeonholes, with n > m, then at least one pigeonhole must contain more than one item.

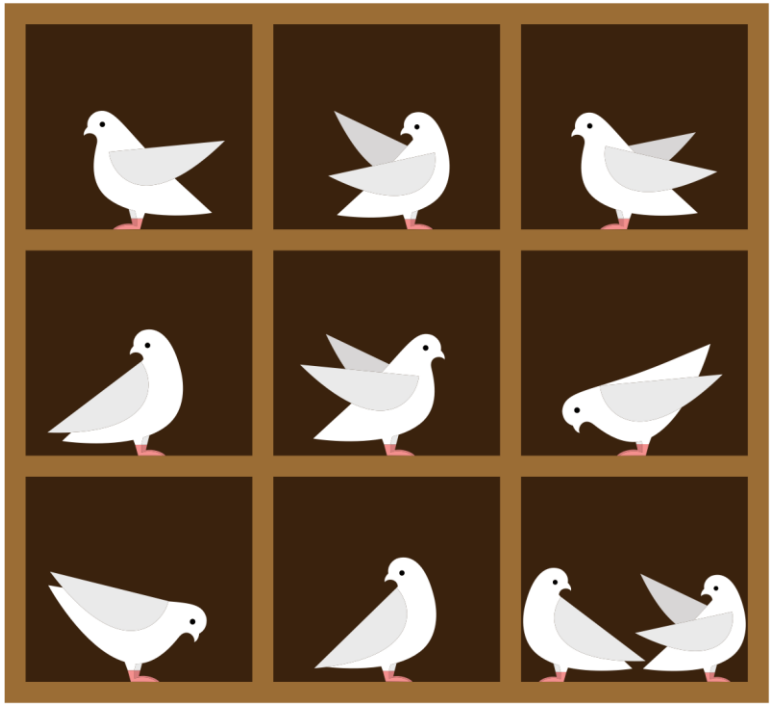


Pigeonhole Principle

A probabilistic generalization of the pigeonhole principle states that if n pigeons are randomly put into m pigeonholes with uniform probability 1/m, then at least one pigeonhole will hold more than one pigeon with probability

$$1 - \frac{(m)_n}{m^n},$$

where $(m)_n$ is the falling factorial $m(m-1)(m-2)\ldots(m-n+1)$.



Collision

Two keys may hash to the same slot in the hash table.

1. Avoid collisions: choosing a suitable hash function.

    ⇒ Impossible to avoid, but minimize collisions

1. Collision resolution techniques.

    Chaning

    Open Addressing

Load Factor

The load factor is a fraction that represents the number of elements stored in the table divided by the size of the table's array

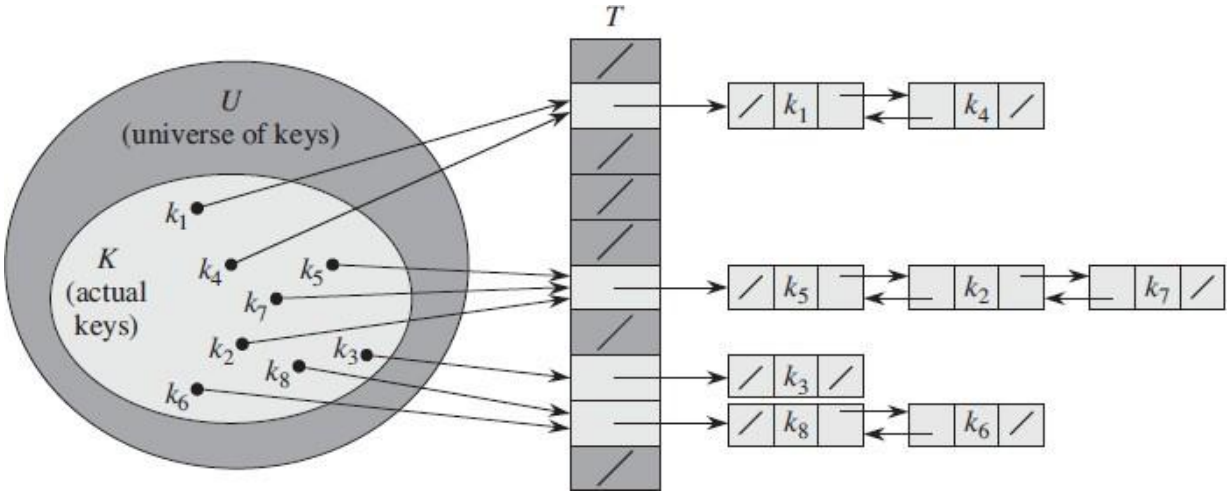
$$\alpha = \frac{\text{The number of elements stored in the table}}{\text{The size of the table's array}}$$

Chaining

Chaining

Simplest technique for collision resolution.

Place all elements that hash to the same slot into the same link list.



Exercise:

Insert the keys 5, 28, 19, 15, 20, 33, 12, 17, 10 into a hash table with collisions resolved by chaining. Let the table have 9 slots, and hash function be h(k) = k mod 9.

0	
1	
2	
3	
4	
5	
6	
7	
8	

Chaining

```
public class HashTable{

        private static int SIZE = 1000
        private LinkedList<String>[] hashtable = (LinkedList<String>[]) new LinkedList[SIZE];

        public void add(String key) {
                int hash = hash(key);
                if( hashtable [hash] == null ){
                                hashtable[hash] = new LinkedList<String>();
                }
                LinkedList<String> bucket = hashtable [hash];
                bucket.add(key);
        }
        ……..
}
```

Chaining - Load Factor

If a hash table T with m slots, holds n elements, load factor (i.e. the average number of elements stored in a chain) of T is :

$$\text{Load factor}(\alpha) = n/m$$

In chaining α can be less than, equal to or greater than 1.

Chaining

Worst-case behaviour: all n keys hashes to the same slot: one linked list of n stores all keys.

- Time for searching: proportional to n.

Average-case: depends on how well hashing algorithm distributes set of keys over m slots, on average

- **Simple uniform hashing**: any given elements is equally likely to hash into any of the m slots, independently of where any other element has hashed to.

Chaining

In a hash table in which collisions are resolved by chaining, and under the assumption of simple uniform hashing:

- An unsuccessful search (element is not in the linked list) or a successful search (element in the list) both takes: average-case time $\Theta(1+\alpha)$

  (α is the load factor., Time takes to search through the linked list).

If m (the number of hash-table slots) is at least proportional to n (the number of elements in the table), then searching tables constant time on average O(1).

Therefore, we can support all dictionary operations: insert, search, delete operations in O(1) time on average.

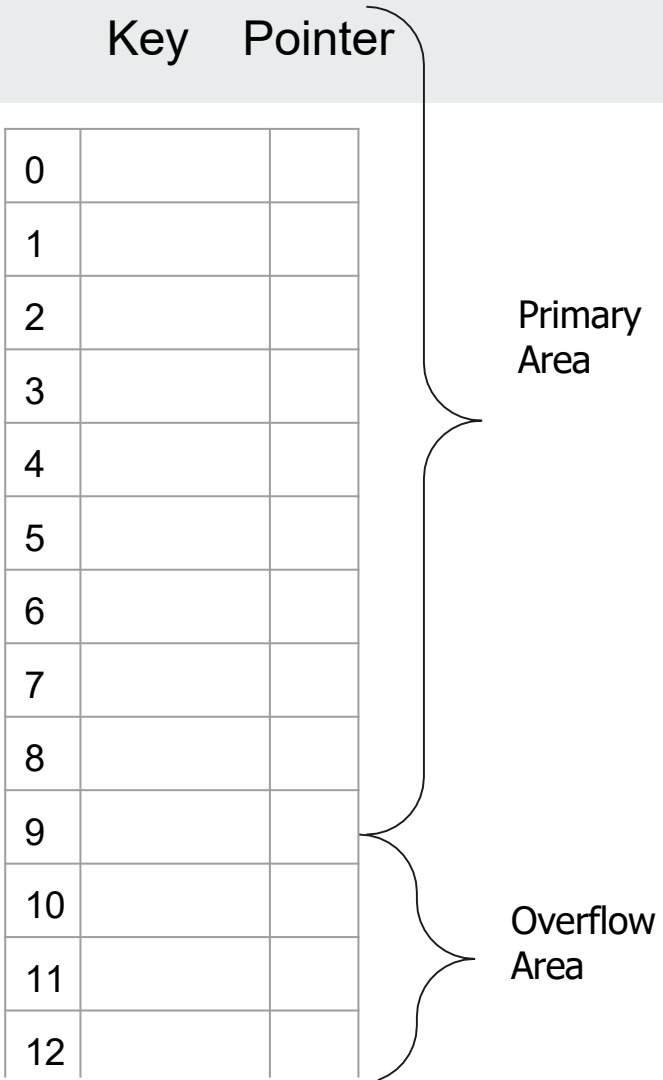
Chaining - Overflow Area

Another scheme will divide the pre allocated table into two sections the primary area to which keys are mapped and an area for collisions, normally termed the overflow area.

Example:

keys 5, 28, 19, 15, 20, 33, 12, 17, 10

hash function be h(k) = k mod 9



Chaining - Overflow Area

Exercise: write the code for changing with overflow area

```
public class HashTable{

        private static int SIZE = 1000
        private LinkedList<String>[] hashtable = (LinkedList<String>[]) new LinkedList[SIZE];

        public void add(String value) {
                int hash = hash(value);
                if( hashtable [hash] == null ){
                        hashtable[hash] = new LinkedList<>();
                }
                LinkedList<String> bucket = hashtable [hash];
                bucket.add(value);
        }
        ……..
}
```

Chaining - Overflow Area

When a collision occurs, a slot in the overflow area is used for the new element and a link from the primary slot established as in a chained system

This is essentially the same as chaining, except that the **overflow area is pre allocated** and thus possibly faster to access

The **maximum number of elements must be known in advance**, but in this case, two parameters must be estimated the optimum size of the primary and overflow areas

Open Addressing

Open Addressing

All elements are stored in the hash table itself

When a key collides with another key, the collision is resolved by finding an available table entry other than the position (to which the colliding key is originally hashed.

Load factor can never exceed 1.

Avoid pointers:

- Instead compute the sequence of slots to be examined.
- Extra memory freed by not storing pointers→ can have larger number of slots with same amount of memory.
- Fewer collisions and faster retrieval.

Open Addressing

Probe: successively examine the hash table until an empty slot is found to insert the key.

Sequence of probe depends on the key being inserted.

Hash function is extended to include the probe number (starting from 0) as a second input.

Open Addressing - Linear Probing

Given an ordinary hash function $h' : U \rightarrow \{0, 1, \ldots, m-1\}$, which is called as an auxiliary hash function, linear probing uses the hash function:

$$h(k, i) = (h'(k) + i) \bmod m$$

For every key k the probe sequence $\langle h(k, 0), h(k, 1), \ldots, h(k, m)\rangle$ is considered. If no free position is found in the sequence the hash table overflows.

Exercise:

Insert the keys 5, 28, 19, 15, 20, 33, 12, 17, 10 into a hash table with collisions resolved by linear probing. Let the table have 9 slots, and auxiliary hash function be h'(k) =k mod 9.
H(k,i) = ((k mod 9)+ i) mod 9

0	
1	
2	
3	
4	
5	
6	
7	
8	

Open Addressing

```
public class OpenAddressingHashTable{
private static int SIZE = 1000;

private String[] hashtable = new String[SIZE];

public void add(String value) {

        int probe = 0;

        do{

                int hash = hash(value, probe);

                 if( hashtable [hash] == null ){

                        hashtable [hash] = value;

                        return;
                        }
                        Probe++;
    } while (probe<SIZE);
    //hashtable is full
    Throw new RuntimeException("has table overflow");
```

Open Addressing - Linear Probing

Easy to implement.

Suffers from primary clustering:

- Clusters arise because an empty slot preceded by i full slots gets filled next [with probability (i+1)/m].
- Long runs of occupied slots tend to get longer, and the average search time increases.

Because the initial probe determines the entire probe sequence, there are only m distinct probe sequences.

Coalesced Chaining

Combines the linear probing with chaining.

Index of the next key position is stored with the key already in the table

Example:

Insert the keys 5, 28, 19, 15, 20, 33, 12, 17, 10

0		
1		
2		
3		
4		
5		
6		
7		
8		

Open Addressing - Quadratic Probing

$$h(k,i) = (h'(k) + c_1 i + c_2 i^2) \bmod m$$

Example:

$$h(k,i) = ((k \bmod m) + c_1 i + c_2 i^2) \bmod m$$

Where h′ is auxiliary hash function, c_1 and c_2 are positive auxiliary constants and $i = 0, 1, \ldots, m - 1$

Initial position: ??

Later positions probed are offset by amounts that depend in a quadratic manner on the probe number.

Exercise:

$$h(k,i) = ((k \bmod m) + c_1 i + c_2 i^2) \bmod m$$

Insert the keys 5, 28, 19, 15, 20, 33, 12 into a hash table with collisions resolved by quadratic probing. Let the table have 9 slots, and auxiliary hash function be h′(k) = k mod 9.

0	
1	
2	
3	
4	
5	
6	
7	
8	

Open Addressing - Quadratic Probing

Works better than linear probing

In order to make full use of the hash table, the values of c_1, c_2 and m are constrained.

If two keys have same initial probe position, then the probe sequence are same → leads to secondary clustering.

The initial probe determines the entire sequence, and so only m distinct probe sequences are used.

Open Addressing - Double Hashing

$$h(k,i) = (h_1(k) + i * h_2(k)) \bmod m$$

Where both h_1 and h_2 are auxiliary hash functions.

Initial probe position: ??

Successive probe positions are offset from previous position by the amount of $h_2(k)$, modulo m.

Open Addressing - Double Hashing

Designing auxiliary functions

- Let m be a power of 2 and h_2 always produce an odd number

  or

- Let m be a prime and h_2 always produce a positive integer less than m
  - Example:

    $h_1(k) = k \bmod m$

    $h_2(k) = 1 + (k \bmod m')$ where m' is slightly less than m (say m - 1)

Exercise:

Insert the keys 18, 41, 22, 44, 59, 32, 31, 73 into a hash table with collisions resolved by double hashing. Let the table have 13 slots, and auxiliary hash functions be:

$$h_1(k) = k \bmod 13 \text{ and } h_2(k) = 8 - (k \bmod 8)$$

$$h(k, i) = [h_1(k) + i * h_2(k)] \bmod m$$

0	1	2	3	4	5	6	7	8	9	10	11	12

Open Addressing

Assuming uniform hashing (each possible probe sequence is equally likely)

- Unsuccessful Search

Expected no. of probes required to find an empty location is at most $\{1/(1-\alpha)\}$

- Successful Search

Expected no. of probes in a successful search is at most $\{(1/\alpha) \ln [1/ (1-\alpha)]\}$

- Insert an element

Expected no of probes on average, at most $\{1/(1-\alpha)\}$

Open Addressing

Load factor can never exceed 1.

Avoid pointers:

- Instead compute the sequence of slots to be examined.
- Extra memory freed by not storing pointers→ can have larger number of slots with same amount of memory.
- Fewer collisions and faster retrieval.

Deletion is difficult

Hash Table Organization

Advantages

Chaining : Unlimited number of elements.

Open addressing : Fast access through use of main table space

Overflow area: Fast access.

Disadvantages

Chaining: Overhead of multiple linked lists

Open addressing : Maximum number of elements must be known; Clusters.

Overflow area: Two parameters which govern performance need to be estimated

Perfect Hashing

Perfect Hashing

Consider a static set of keys:
Once we store the values they do not change (no insertions or deletions)

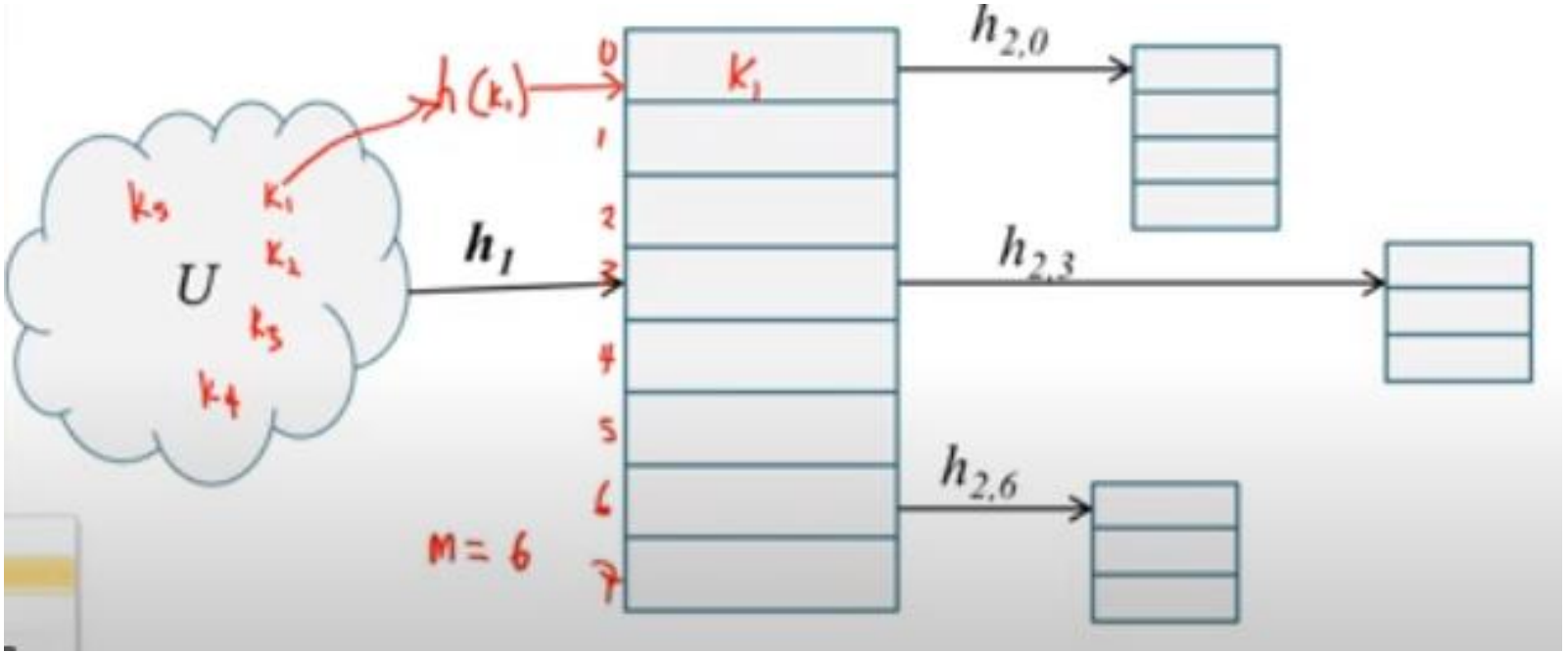
Examples:

- Files in a CD-ROM
- Reserved keywords of a programming language

Perfect Hashing

- Perfect Hashing is a hashing technique that requires O(1) memory accesses to perform a search in the worst case.
- A perfect hashing scheme is created with a two level hashing scheme with universal hashing at each level.
- First Level:
  - Same as hashing with chaining
- Second Level:
  - When a key hashed to slot j, a secondary hash table S_j is created at j selecting a hash function h_j guaranteeing that no collisions occur at the secondary level.

Perfect Hashing Structure



Self Study Exercise

Applications of Hashing

Dictionary, spell checker: lookup

Databases

Cryptography

Compilers: symbol tables

Games: lookup board to find the move (chess, tic tac toe)

UNIX shell: quick command lookup

Thank You!!